

Name: Nivetha Dhanakoti
Reg no.: 3122 22 5001 087

To be proficient in developing an application to simulate flow control technique using socket programming in C

Write the code using socket programming in C

1. Establish the basic connection between client and the server.
2. Develop an application to simulate flow control technique using HDLC protocol.
3. Client gets input data from user and divides them as 4 bits into a packet.
4. Add header with sequence number, source and destination address.
5. Send them to server. Before sending each packet ask whether to send or not (to create loss).
6. Server on receiving the packet and by reading the sequence number, acknowledges the client as per Go Back N Protocol.
7. If any packet is missing, based on the acknowledgement received from server, client retransmits it.
8. On Client side, before beginning new transmission, asks whether to close connection.

Sample Input and Output

Client:	Server:
Data:00101001	
Packets:0010 1001	
2 packets	
Enter source address: A	
Enter destination address: B	
Send or Not: Y	
Packet1: F-B-0-0010-FCS-F (Flag-Destination-SeqNo-Data-FCS-Flag)	
Send or Not: Y	Received: F-B-0-0010-FCS-F
Sent Packet1	
Next Packet:	
Send or Not: Y	
Packet 2: F-B-1-1001-FCS-F	
Sent Packet2	Received: F-B-1-1001-FCS-F
	Sending Acknowledgment: ACK2
Received Acknowledgement: ACK2	
End Transmission: No	
Data:1111010111011011	
Packets: 1111 0101 1101 1011	
4 packets	
Enter source address: A	
Enter destination address: B	
Send or Not: Y	
Packet1: F-B-0-1111-FCS-F	
Send or Not: Y	
Sent Packet1	


```

// Create socket
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}

// Bind the socket to the port
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

// Listen for incoming connections
if (listen(server_fd, 3) < 0) {
    perror("listen");
    exit(EXIT_FAILURE);
}

// Accept the connection from the client
if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen)) <
0) {
    perror("accept");
    exit(EXIT_FAILURE);
}

while (1) {
    // Clear the buffer before reading
    memset(buffer, 0, sizeof(buffer));

    // Read the data sent by the client
    int valread = read(new_socket, buffer, sizeof(buffer));
    if (valread > 0) {
        printf("\nReceived: %s\n", buffer);

        // Extract sequence number from the packet (assuming the sequence number is in the
correct position)
        char seq_num = buffer[6] - '0';

        // Send an acknowledgment for the received packet
        send_acknowledgment(new_socket, seq_num);
    } else if (valread == 0) {
        printf("\nConnection closed by client.\n");
        break;
    } else {
        perror("recv");
    }
}

close(new_socket);
close(server_fd);

```

```
    return 0;
}
```

Client Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8888

// Function to send a packet with flow control
int send_packet(int sock, char* packet, int seq_num) {
    char buffer[1024] = {0};

    printf("\nPacket%d: %s\n", seq_num, packet);
    char decision;
    printf("Send or Not (Y/N): ");
    scanf(" %c", &decision);

    if (decision == 'Y' || decision == 'y') {
        send(sock, packet, strlen(packet), 0);
        printf("Sent Packet%d\n", seq_num);
        read(sock, buffer, sizeof(buffer));
        printf("Received Acknowledgement: %s\n", buffer);
        return 1; // Packet was sent successfully
    } else {
        return 0; // Packet was not sent
    }
}

// Function to create a packet with sequence number, source, destination, and data
void create_packet(char* packet, char* src_addr, char* dest_addr, int seq_num, char* data) {
    sprintf(packet, "F-%s-%d-%s-FCS-F", dest_addr, seq_num, data);
}

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char data[1024];
    char src_addr[10], dest_addr[10];
    int packets_sent[100] = {0}; // Keep track of sent packets, 0 = not sent, 1 = sent

    // Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 and IPv6 addresses from text to binary form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }

    // Connect to the server
```

```

if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("\nConnection Failed \n");
    return -1;
}

while (1) {
    printf("\nData: ");
    scanf("%s", data);

    int len = strlen(data);
    int total_packets = (len + 3) / 4; // Calculate number of 4-bit packets

    printf("\nPackets: ");
    for (int i = 0; i < len; i += 4) {
        for (int j = i; j < i + 4 && j < len; ++j) {
            printf("%c", data[j]);
        }
        printf(" ");
    }
    printf("\n%d packets\n", total_packets);

    printf("Enter source address: ");
    scanf("%s", src_addr);

    printf("Enter destination address: ");
    scanf("%s", dest_addr);

    // Send packets
    for (int i = 0; i < total_packets; i++) {
        if (packets_sent[i] == 0) { // Only send packets that haven't been sent
            char packet[1024] = {0};
            char chunk[5] = {0};

            strncpy(chunk, &data[i * 4], 4); // Extract 4-bit chunk
            create_packet(packet, src_addr, dest_addr, i, chunk);

            if (send_packet(sock, packet, i)) {
                packets_sent[i] = 1; // Mark packet as sent
            }
        }
    }

    // Retransmit unsent packets if any
    for (int i = 0; i < total_packets; i++) {
        if (packets_sent[i] == 0) { // Check for unsent packets
            printf("\nRetransmitting:\n");
            char packet[1024] = {0};
            char chunk[5] = {0};

            strncpy(chunk, &data[i * 4], 4); // Extract 4-bit chunk
            create_packet(packet, src_addr, dest_addr, i, chunk);

            send_packet(sock, packet, i); // Attempt to resend
        }
    }

    char end_transmission[5];
    printf("\nEnd Transmission (Yes/No): ");
    scanf("%s", end_transmission);
}

```

```

        if (strcmp(end_transmission, "Yes") == 0 || strcmp(end_transmission, "yes") == 0) {
            close(sock);
            printf("Transmission Ended\n");
            break;
        }

        // Clear buffer for new input
        memset(data, 0, sizeof(data));
        memset(packets_sent, 0, sizeof(packets_sent)); // Reset sent packets tracking
    }

    return 0;
}

```

Server Output:

```

nivethadhanakoti@Nivethas-MacBook-Pro A8 % ./server

Received: F-B-0-0010-FCS-F
Sending Acknowledgment: ACK0

Received: F-B-1-1001-FCS-F
Sending Acknowledgment: ACK1

Received: F-B-0-1111-FCS-F
Sending Acknowledgment: ACK1

Received: F-B-1-0101-FCS-F
Sending Acknowledgment: ACK0

Received: F-B-3-1011-FCS-F
Sending Acknowledgment: ACK1

Received: F-B-2-1101-FCS-F
Sending Acknowledgment: ACK1

Connection closed by client.

```

Client Output:

```

nivethadhanakoti@Nivethas-MacBook-Pro A8 % ./client

Data: 00101001

Packets: 0010 1001
2 packets
Enter source address: A
Enter destination address: B

Packet0: F-B-0-0010-FCS-F
Send or Not (Y/N): Y
Sent Packet0
Received Acknowledgement: ACK0

Packet1: F-B-1-1001-FCS-F
Send or Not (Y/N): Y
Sent Packet1
Received Acknowledgement: ACK1

End Transmission (Yes/No): No

```

```
Data: 1111010111011011

Packets: 1111 0101 1101 1011
4 packets
Enter source address: A
Enter destination address: B

Packet0: F-B-0-1111-FCS-F
Send or Not (Y/N): Y
Sent Packet0
Received Acknowledgement: ACK1

Packet1: F-B-1-0101-FCS-F
Send or Not (Y/N): Y
Sent Packet1
Received Acknowledgement: ACK0

Packet2: F-B-2-1101-FCS-F
Send or Not (Y/N): N

Packet3: F-B-3-1011-FCS-F
Send or Not (Y/N): Y
Sent Packet3
Received Acknowledgement: ACK1

Retransmitting:

Packet2: F-B-2-1101-FCS-F
Send or Not (Y/N): Y
Sent Packet2
Received Acknowledgement: ACK1

End Transmission (Yes/No): Yes
Transmission Ended
```

Learning Outcomes:

I have learnt to be proficient in developing an application to simulate flow control technique using socket programming in C